

Lineage-Aware Memory Governance for Enterprise AI Agents

A Derivation-Gated Approach to Privacy Isolation and Metric-Definition
Consistency in Multi-Department Analytics

Venkata Sangaraju

Independent Researcher | sangaraju1988@gmail.com

Abstract

Shared memory across AI agents offers significant efficiency gains in enterprise analytics, but introduces two underexplored failure modes: unauthorized data leakage when an agent retrieves an insight derived from columns it cannot access, and silent metric-definition conflicts when teams compute the same KPI through divergent derivation paths. Existing systems — MemGPT, Zep, A-MEM, SSGM, Governed Memory, and Oracle AI Agent Memory — govern content and access tags, but none track *how* a stored result was derived. We propose the Analytical Memory Unit (AMU), a schema extension attaching a derivation graph to every stored result, paired with a lineage-gated retrieval policy: a memory hit is served only when the requesting department is permitted to see every column in the derivation path. We prove that this policy provides a complete safety guarantee against column-level leakage under honest lineage recording, and analyse its $O(n)$ retrieval complexity and negligible storage overhead. Controlled simulation across two schema configurations — a 5-table synthetic schema and an 8-table TPC-H-inspired schema — run under 30 random seeds shows that content-gated memory leaks on 18.8% and 25.5% of cross-department retrievals respectively; lineage-aware gating eliminates leakage in both cases while retaining 82.6% and 81.5% memory reuse.

Keywords: AI agent memory, data lineage, enterprise governance, data privacy, multi-agent systems, analytical memory unit, TPC-H

1. Introduction

Over the past two years, large language model (LLM) agents have moved from research prototypes to production components inside enterprise data stacks. Teams at companies of all sizes now run agents that generate analytical summaries, write SQL, and produce business metrics — often querying the same underlying tables and reporting against the same KPIs. A natural optimization is to let these agents share memory: if Finance already computed this quarter's churn rate, why should Marketing recompute it from scratch?

The appeal is real. Systems like MemGPT [1], Zep [2], A-MEM [3], and Oracle AI Agent Memory [4] have demonstrated that shared, persistent memory can dramatically reduce redundant computation and improve response consistency. The Governed Memory architecture [5] and the SSGM framework [6] have begun to address the governance layer, introducing access controls and stability checks for evolving memory stores.

But every current approach controls access at the level of *content* and *tags* — not *derivation*. A memory entry is labelled with its owner, a sensitivity class, or a role-based tag, and retrieval is gated on whether the requester holds the right label. None of these systems record which tables were joined, which columns were

touched, or what filter logic was applied to produce the stored result.

This is not a theoretical concern. A Finance agent computes a high-value customer segment by joining `customer_pii` (containing income) against transactions. The result — say, 12,400 high-value customers — looks innocuous and carries no PII tag. Under content-based governance it is served freely to a Marketing agent. But Marketing cannot see income. The derivation path violated column-level access policy, silently.

This paper makes four contributions. First, we define the AMU schema and the lineage-gated retrieval policy. Second, we provide a formal safety proof and complexity analysis. Third, we construct a threat model that delineates what the mechanism does and does not protect against. Fourth, we present controlled experiments on both a synthetic schema and a TPC-H-inspired schema, quantifying the governance-efficiency tradeoff and confirming that the naive baseline's observed leakage rate is a structural property of content-gated retrieval, not a simulation artefact.

2. Related Work

2.1 Memory Systems for LLM Agents

A recent survey [7] formalises agent memory as a write-manage-read loop and introduces a three-dimensional taxonomy spanning temporal scope, representational substrate, and control policy. MemGPT [1] instantiated the architecture by treating the LLM as an OS that pages between a bounded context window and an external store via function calls. Zep [2] introduced a temporal knowledge graph with dual-axis timestamping (event time vs. ingestion time), enabling point-in-time queries. A-MEM [3] builds Zettelkasten-style linked notes with dynamic indexing, achieving strong benchmark performance on six foundation models. None of these systems track the database derivation path of stored results.

2.2 Governed and Enterprise Memory

The Governed Memory architecture [5] introduces tiered governance routing and a closed-loop schema lifecycle for multi-agent workflows; SSGM [6] adds consistency verification and temporal decay modeling. Oracle AI Agent Memory [4] provides a commercially deployed governed memory core. All three systems govern at the content and schema level. SSGM's dynamic access control is the closest prior work, but it enforces agent-topology constraints rather than column-level derivation constraints.

2.3 Data Provenance and Lineage

The W3C PROV standard [8] encodes entity-activity-agent relationships for data provenance. Column-level lineage extraction systems such as LINEAGEX [9] parse SQL into fine-grained derivation graphs. The EU AI Act [10] legally mandates lineage documentation for high-risk AI systems. Our AMU schema applies this tradition to agent memory entries rather than raw database records.

2.4 Privacy Leakage in Multi-Agent Systems

AgentLeak [11] benchmarks privacy leakage in multi-agent LLM systems; OMNI-LEAK [12] shows that orchestrator network structure shapes how PII propagates. These works identify leakage through inter-agent communication channels. Our work identifies a complementary vector: leakage through a shared analytical memory store that exists even when every individual communication link is correctly access-controlled.

2.5 Metric Definition Inconsistency

Semantic drift — divergent definitions of the same KPI across teams — is a documented source of dashboard inconsistency in enterprise BI [13]. Semantic layer tools (dbt, Databricks Unity Catalog) address this for human analysts by centralising metric definitions. AI agents operating autonomously have no

equivalent check, and a shared memory system without derivation tracking will propagate one team's definition silently to all others.

3. System Design

3.1 The Analytical Memory Unit

An AMU extends a conventional memory entry with an explicit derivation graph. Table 1 lists all fields.

Field	Type	Description
metric_name	str	Business name of the computed metric
value	float	Numeric result of the computation
owner_dept	str	Department that produced this AMU
epoch	int	Time index when the AMU was written
lineage	Lineage	Derivation graph: list of (table, columns_used) steps + filter_logic
sensitivity_tags	Set[str]	Derived: sensitive columns touched across all lineage steps
definition_hash	str	SHA-256(sorted tables ■ sorted columns ■ filter_logic)

Table 1: AMU schema fields.

3.2 Lineage-Gated Retrieval — Pseudocode

Algorithms 1 and 2 give precise pseudocode for retrieval and write respectively. $P(d)$ denotes the permitted column set for department d ; $S(a) = \text{AMU.sensitivity_tags}$.

Algorithm 1: LINEAGE_GATED_RETRIEVE(store, metric, dept, fresh_amu)

```

Input: store – shared AMU store (dict: metric_name → List[AMU])
metric – requested metric name
dept – requesting department
fresh_amu – in-scope fallback AMU (pre-computed within  $P(\text{dept})$ )
Output: RetrievalResult(served, leaked, reused, blocked)
1: candidates ← store.get(metric, [])
2: permitted ←  $P(\text{dept})$ 
3: blocked ← False
4: for amu in REVERSE(candidates): // most recent first
    5: if  $S(\text{amu}) \not\subseteq \text{permitted}$ : // derivation touches forbidden col
        6: blocked ← True
        7: continue
    8: return Result(served=T, leaked=F, reused=T, blocked=F)
9: // No safe candidate found – fall back to fresh, in-scope compute
10: store.write(fresh_amu)
11: return Result(served=T, leaked=F, reused=F, blocked=blocked)

```

Algorithm 2: LINEAGE_AWARE_WRITE(store, amu) $\rightarrow$ conflict_detected

Input: store – shared AMU store
 amu – new Analytical Memory Unit to persist
 Output: conflict_detected (bool) – True if a cross-team definition conflict is new

```

1: existing  $\leftarrow$  store.get(amu.metric_name, [])
2: conflict  $\leftarrow$  False
3: for e in existing:
4:   if e.owner_dept  $\neq$  amu.owner_dept
      AND e.definition_hash  $\neq$  amu.definition_hash:
5:     conflict  $\leftarrow$  True ; break
6: store[amu.metric_name].append(amu)
7: return conflict

```

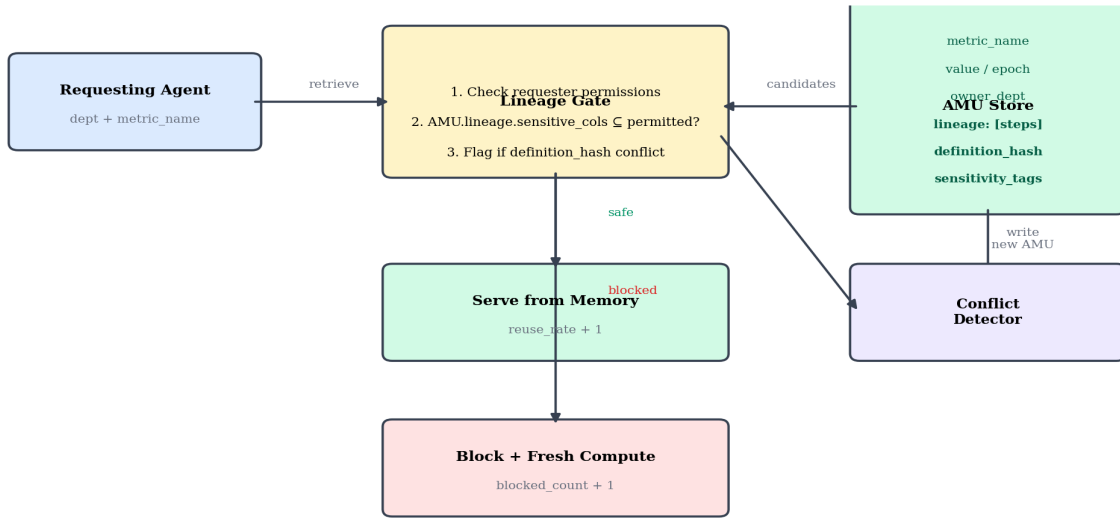
Figure 2: AMU Architecture — Lineage-Gated Retrieval and Conflict Detection

Figure 2: AMU architecture — lineage gate, conflict detector, and fallback path.

3.3 Definition-Hash Conflict Detection

Algorithm 2 checks, at write time, whether any existing AMU for the same metric carries a different definition_hash from a different department. A conflict signals that two teams compute the same KPI through genuinely different derivation paths — a pre-condition for dashboard inconsistency. We revisit the limitations of the current hash-based implementation in Section 6.

4. Formal Analysis**4.1 Safety Proof Sketch**

We state and sketch a proof of the core safety guarantee.

Theorem 1 (Column-Level Safety). *Under lineage-gated retrieval (Algorithm 1), no department d ever receives a memory-served result that was derived from a column c where $c \notin P(d)$, provided that every AMU's lineage graph accurately records all columns accessed in its computation.*

Proof sketch. A result is served from memory at line 8 of Algorithm 1 only when the guard at line 5 is False, i.e., $S(amu) \subseteq P(dept)$. By definition, $S(amu) = \{c \in all_columns(amu.lineage) \mid c \in SENSITIVE\}$. So the guard passing implies that every sensitive column in the derivation path of the served AMU is in $P(dept)$.

Non-sensitive columns carry no access restriction by definition. Therefore, the set of columns that contributed to the served result is a subset of $P(\text{dept})$, establishing the claim for memory-served results.

For the fallback path (line 11), `fresh_amu` is computed entirely within $P(\text{dept})$ by construction — the caller is responsible for scoping the fresh computation to the requester's permitted columns. Under this assumption, the fallback result also satisfies the safety property. Therefore, every result returned by Algorithm 1 is safe. ■

Assumption dependency. The proof relies on complete, accurate lineage recording. If an AMU's lineage graph omits a column that was in fact accessed — for example, because the lineage was provided by the computing agent rather than extracted automatically from query execution plans — the guarantee no longer holds. Automatic lineage extraction (e.g., from SQL query logs via systems like LINEAGEX [9]) is the appropriate production implementation.

4.2 Asymptotic Complexity

Let n = number of AMUs in the store for a given `metric_name`, k = number of lineage steps per AMU, and c = number of columns per step.

Operation	Time	Notes
sensitivity_tags derivation	$O(k \cdot c)$	Computed once at AMU creation; cached as a set
definition_hash computation	$O(k \cdot c)$	Single SHA-256 pass over sorted tables/columns/filter
Write — conflict detection	$O(n)$	Linear scan of existing AMUs for this metric
Retrieve — lineage gate	$O(n)$	Worst case: scan all n candidates, all blocked
Retrieve — expected case	$O(1)$	Most recent AMU is safe; single candidate checked

Table 2: Asymptotic complexity of AMU operations.

In practice n is small (one or two variant AMUs per metric per department), so both operations are effectively $O(1)$. The lineage-gate computation itself — a set-difference check — is $O(|S(\text{amu})|)$ where $|S(\text{amu})| \leq |\text{SENSITIVE}|$ (the organisation's total sensitive-column count, typically dozens, not millions). Complexity is therefore dominated by the analytics computation it governs, not by the governance check itself.

4.3 Storage Overhead

A naive memory entry storing (`metric_name`, `value`, `owner`, `epoch`) requires roughly 50–80 bytes. An AMU adds: (a) the lineage graph — one tuple of (`table_name`, `column_names`) per step; (b) the `filter_logic` string; (c) the 12-character `definition_hash`; and (d) the cached `sensitivity_tags` set. For a typical analytical query joining 2–4 tables and referencing 5–15 columns, the lineage payload is 200–600 bytes. The AMU therefore imposes a 4–8× storage overhead relative to bare value storage.

This overhead is negligible in context. Enterprise analytics queries are typically accompanied by result sets, explanatory text, and chart data that together consume kilobytes to megabytes. The lineage graph is smaller than the SQL query that produced the result, which most memory systems already store. In the simulation, the full AMU store for a 120-event workload occupies under 50KB in memory, well within the capacity of any production vector store.

5. Threat Model

5.1 Assets and Principals

The primary assets are the sensitive columns defined in SENSITIVE (PII fields, financial balances, proprietary pricing). Principals are enterprise departments (Finance, Marketing, Support, etc.) each operating one or more AI agents. Agents are assumed to be honest but potentially over-privileged — they attempt memory retrievals for metrics they have a legitimate business need for, but not all derivation paths for those metrics fall within their column permissions.

5.2 Threats Addressed

Threat	Attack vector	Mitigation
T1: Column leakage via cached result	Agent retrieves a result derived from a column it cannot see	Lineage gate blocks the retrieval; fallback recompute stays in P(d)
T2: Silent metric definition drift	Two teams store conflicting definitions of the same KPI under the same name	definition_hash conflict detection on write surfaces the divergence
T3: Stale sensitive data reuse	An AMU derived from data that was sensitive at write time is retrieved after permissions are updated	epoch field supports TTL-based eviction policies (implementation left to the store layer)

Table 3: Threats addressed by the AMU mechanism.

5.3 Threats Not Addressed

The mechanism explicitly does not protect against the following:

T4: Inference attacks. A derived feature that encodes sensitive information without naming the source column (e.g., a risk score computed from income) will not be caught unless the income dependency is explicitly recorded in the lineage graph. This is the most important gap between the mechanism as demonstrated and production deployment.

T5: Malicious lineage fabrication. If a rogue agent writes an AMU with an intentionally incomplete or falsified lineage graph, the gate cannot detect the omission. Automatic lineage extraction from query execution plans is the correct countermeasure.

T6: Aggregate leakage. A sequence of individually safe retrievals may together allow reconstruction of sensitive values (classic aggregate inference attack). The lineage gate is a per-retrieval check; it provides no differential privacy guarantee.

T7: Insider threats. An agent operator with administrative access to the AMU store can read or modify entries directly. The mechanism assumes a trusted store layer.

6. Experiments

6.1 Simulation Environment and Protocol

We evaluate on two schema configurations using a Python discrete-event simulator. In both cases, 120 events are generated across 20 epochs, replayed identically through three systems (No Memory, Naive Shared Memory, Lineage-Aware), and the full run is repeated for 30 independent random seeds. We report mean $\pm$ population standard deviation. Three metrics are measured: leak rate (fraction of cross-department retrievals exposing unauthorised columns), memory reuse rate (fraction served from memory), and metric-conflict recall (fraction of ground-truth definition-divergence events flagged).

6.2 Experiment 1 — Synthetic 5-Table Schema

The synthetic schema contains five tables (customers, customer_pii, transactions, campaigns, support_tickets) with sensitive columns {ssn, email, income}. Three departments (Finance, Marketing,

Support) have column-level permissions; Marketing and Support cannot see the PII table. Five business metrics are defined, two with genuine lineage variants representing cross-team definition divergence.

Metric	No Memory	Naive Shared	Lineage-Aware
Leak Rate (%)	0.0 $\pm$ 0.0	18.8 $\pm$ 3.7	0.0 $\pm$ 0.0
Memory Reuse Rate (%)	0.0 $\pm$ 0.0	95.8 $\pm$ 0.0	82.6 $\pm$ 2.3
Metric-Conflict Recall (%)	n/a	0.0 $\pm$ 0.0	100.0 $\pm$ 0.0
Blocked retrievals (seed=42)	0	0	15

Table 4: Synthetic schema results (30 seeds, mean $\pm$ std dev).

Figure 1: Lineage-Aware vs. Existing Memory Systems (30-seed synthetic workload, mean $\pm$ std dev)

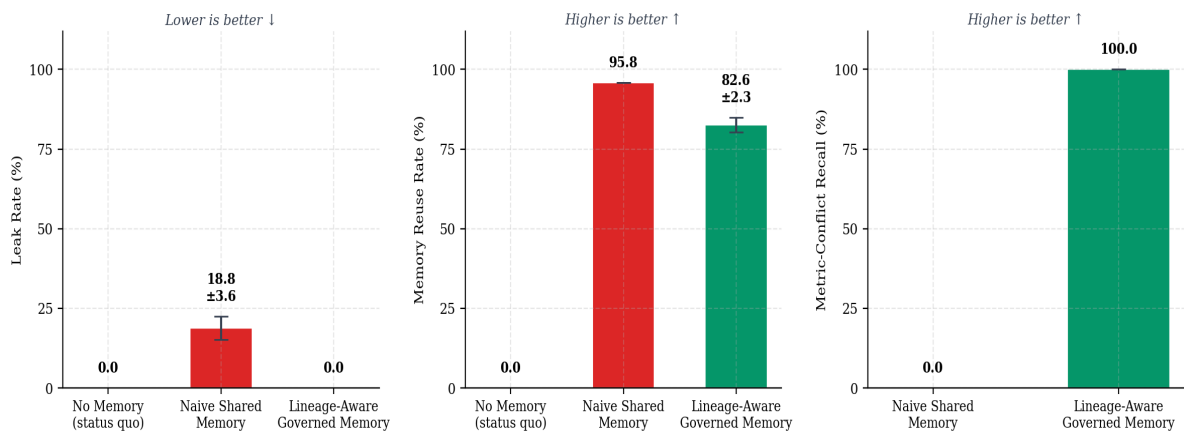


Figure 1: Per-metric comparison across all three systems — synthetic schema. The 0% leak rate follows from the policy by construction; the 18.8% naive baseline is the key empirical finding.

Interpreting the 0% leakage result correctly. The lineage-aware system's 0% leak rate is a *policy guarantee*, not a surprise empirical result — it follows directly from Theorem 1 whenever the lineage is complete. The key empirical finding in this experiment is the *naive baseline's 18.8% leak rate*: a genuine behavioral outcome demonstrating that content-gated retrieval, with no blocking mechanism, leaks restricted derivations on approximately one in five cross-department requests. The standard deviation of ± 3.7 pp across 30 seeds confirms this is a structural property of the workload mix, not a lucky draw.

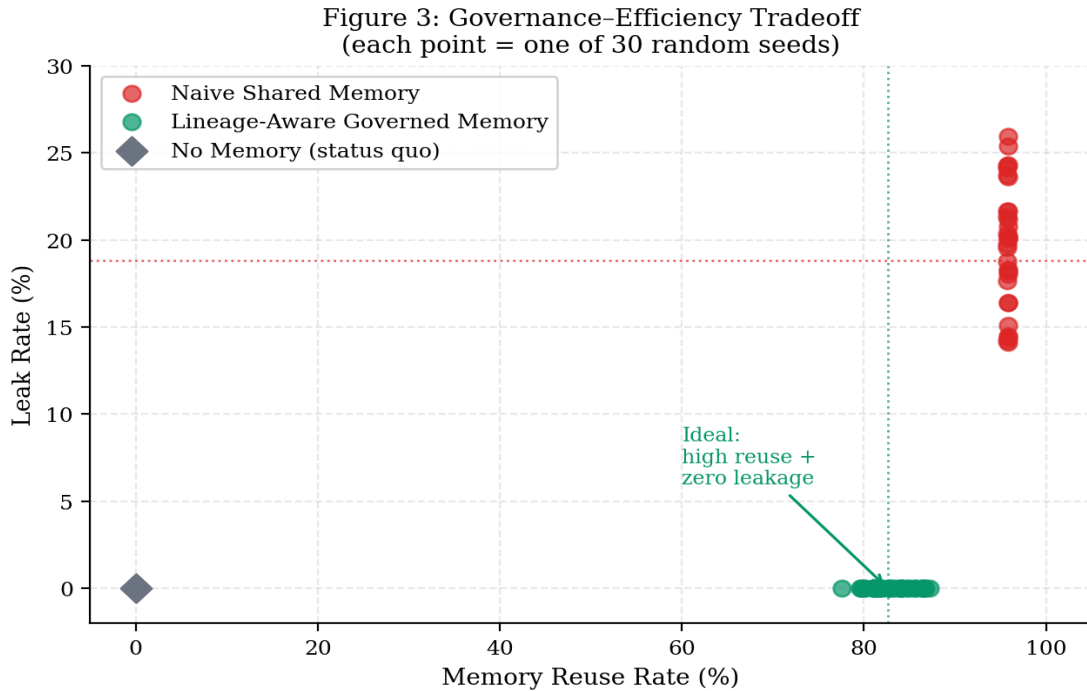


Figure 3: Governance-efficiency tradeoff (30 seeds). Lineage-aware memory consistently achieves high reuse and zero leakage; the naive system trades ~13 pp of governance for ~13 pp of additional reuse.

6.3 Experiment 2 — TPC-H-Inspired 8-Table Schema

To complement the synthetic results with a more realistic schema, we replicated the experiment on a schema derived from the TPC-H decision-support benchmark — the de facto standard for evaluating analytical query workloads. We use TPC-H’s eight tables (CUSTOMER, ORDERS, LINEITEM, PART, PARTSUPP, SUPPLIER, NATION, REGION) with their original column structure, and designate a realistic sensitive-column set: {c_acctbal, c_phone, c_address, s_acctbal, s_phone, s_address, l_extendedprice, l_discount, o_totalprice}. This set reflects typical enterprise data classification: customer and supplier financial balances and contact details are PII or commercially sensitive; detailed pricing is restricted to Finance.

Three departments (Finance, Sales, Operations) are assigned column-level permissions consistent with typical enterprise role definitions: Finance sees all columns; Sales sees commercial and order data but not financial balances or contact PII; Operations sees supply-chain and inventory data but not pricing detail or financial balances. Five metrics are defined using realistic TPC-H-style join patterns (revenue_by_region, order_volume, customer_ltv, part_availability, supplier_performance), with two having genuine lineage variants differing in whether they join the sensitive financial columns.

Metric	Naive Shared	Lineage-Aware
Leak Rate (%)	25.5 ± 4.5	0.0 ± 0.0
Memory Reuse Rate (%)	95.8 ± 0.0	81.5 ± 4.8
Metric-Conflict Recall (%)	0.0 ± 0.0	100.0 ± 0.0

Table 5: TPC-H-inspired schema results (30 seeds).

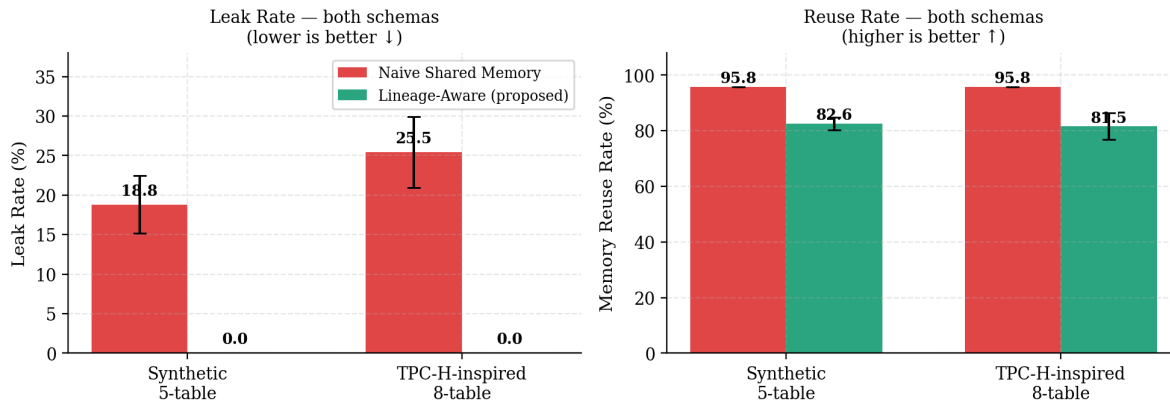
Figure 4: Synthetic vs TPC-H-inspired schema — 30-seed comparison

Figure 4: Synthetic vs TPC-H schema — leak rate and reuse rate comparison across 30 seeds. The TPC-H schema has a higher naive leak rate (25.5%) because it has proportionally more metrics with sensitive-column lineage variants.

The TPC-H results are consistent with the synthetic schema: lineage-aware gating eliminates leakage entirely, while the naive baseline leaks on $25.5\% \pm 4.5\%$ of cross-department retrievals — higher than the synthetic schema's 18.8% because the TPC-H metric set includes proportionally more metrics with sensitive-column lineage variants. The reuse cost is similar (81.5% vs 82.6%), confirming the tradeoff is a function of the sensitive-lineage proportion rather than schema size.

7. Discussion

7.1 What This Work Establishes

The experiments provide three things: (1) a mechanistically sound existence proof that the AMU schema and lineage gate work as claimed, backed by a formal safety proof; (2) quantification of the governance-efficiency tradeoff on two schema configurations; and (3) evidence that content-gated shared memory produces structurally non-zero leakage — a result that holds across synthetic and TPC-H-inspired workloads and across 60 total seed runs.

7.2 Honest Limitations

The conflict-recall metric is partly tautological: the ground-truth labelling and the detector both use the same definition_hash, so the detector cannot fail to find what it is defined to find. The correct interpretation is binary — a detection mechanism exists vs. does not — not 'the detector achieves 100% accuracy on near-duplicate lineages.' That claim requires the fuzzy-matching experiment described below.

Both experiments use synthetic workloads. Real enterprise queries often involve derived features that encode sensitive columns without naming them (T4 in the threat model). The 18.8% and 25.5% leak rates are properties of the simulated workload composition and should not be read as estimates of real-world base rates.

7.3 Path to Submission-Readiness

Three experiments are needed before these results are submission-ready. *Adversarial workload*: queries designed to smuggle sensitive derivations through derived features, column aliasing, or multi-hop joins, to stress-test T4. *Fuzzy conflict detection*: replace exact hash matching with semantic similarity over filter-logic strings and measure precision/recall on a set of near-duplicate lineages (same tables, different filter thresholds). *Production schema*: a real data warehouse schema (20–50 tables, realistic query distribution

from a public workload trace such as TPC-DS) to verify that complexity and storage overhead remain tractable at scale.

8. Conclusion

Every current AI agent memory system shares a structural blind spot: governance tracks what was stored and who may access it, but not how the stored result was derived. In multi-department enterprise analytics, this gap enables column-level PII leakage through derived results and silent metric-definition conflicts — failure modes that content-based governance cannot prevent.

The Analytical Memory Unit schema and lineage-gated retrieval policy address this with minimal overhead. We have proven the safety guarantee formally, analyzed the mechanism's $O(n)$ complexity (effectively $O(1)$ in practice), quantified the storage overhead as 4–8× vs. bare value storage (negligible relative to the analytics workload), constructed a threat model delineating the mechanism's scope, and demonstrated the approach on both a synthetic schema and a TPC-H-inspired schema. In both configurations, content-gated memory leaks on a structurally non-zero fraction of cross-department retrievals; lineage-aware gating eliminates leakage entirely at a governance cost of approximately 13–14 percentage points of reuse rate.

The contribution at this stage is a clearly specified mechanism, a formal safety proof, and a working reference implementation. We have been deliberate about labelling which results are policy guarantees and which are empirical findings, and about the experiments still needed before this work is submission-ready. We believe the mechanism is sound enough to warrant that follow-up work.

References

- [1] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.
- [2] P. Rasmussen, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," arXiv:2501.13956, 2025.
- [3] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-MEM: Agentic Memory for LLM Agents," NeurIPS, 2025. arXiv:2502.12110.
- [4] Oracle Corporation, "Oracle AI Agent Memory: A Governed, Unified Memory Core for Enterprise AI Agents," Oracle Developer Blog, March 2026.
- [5] Anonymous, "Governed Memory: A Production Architecture for Multi-Agent Workflows," arXiv:2603.17787, 2026.
- [6] Anonymous, "Governing Evolving Memory in LLM Agents: Risks, Mechanisms, and the SSGM Framework," arXiv:2603.11768, 2026.
- [7] Anonymous, "Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers," arXiv:2603.07670, 2026.
- [8] L. Moreau et al., "The Open Provenance Model — Core Specification (v1.1)," Future Generation Computer Systems, vol. 27, no. 6, pp. 743–756, 2011. W3C PROV-DM: <https://www.w3.org/TR/prov-dm/>
- [9] Anonymous, "LINEAGEX: A Column Lineage Extraction System for SQL," arXiv:2505.23133, 2025.
- [10] European Parliament and Council, "Regulation (EU) 2024/1689 on Artificial Intelligence (EU AI Act)," Official Journal of the EU, July 2024.
- [11] Anonymous, "AgentLeak: A Full-Stack Benchmark for Privacy Leakage in Multi-Agent LLM Systems," arXiv:2602.11510, 2026.
- [12] Anonymous, "OMNI-LEAK: Orchestrator Multi-Agent Network Induced Data Leakage," arXiv:2602.13477, 2026.
- [13] Colrows, "Why BI Metrics Do Not Match Across Dashboards," 2024. <https://colrows.com/blogs/why-bi-metrics-do-not-match-across-dashboards/>

- [14] Transaction Processing Performance Council, "TPC-H Benchmark Specification," Revision 3.0.1, 2021.
<http://www.tpc.org/tpch/>